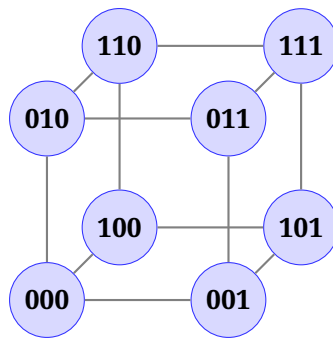

Advanced Computer Architecture

Academic Student Notes

UNIT - I: Theory of Parallelism & Parallel Computer Models



Department of Computer Science and Engineering
Marri Laxman Reddy Institute of Technology and Management
(UGC Autonomous, R25 Regulation)

Compiled: July 14, 2026

Contents

1	Theory of Parallelism & The State of Computing	2
1.1	Architectural Development Tracks	2
1.2	Theoretical Models: PRAM and VLSI	2
2	Parallel Computer Models	3
2.1	Multiprocessors (Shared-Memory Systems)	3
2.2	Multicomputers (Distributed-Memory Systems)	4
2.3	Comparison Matrix	4
2.4	Vector and SIMD Computers	4
3	Program and Network Properties	5
3.1	Conditions of Parallelism	5
3.2	Types of Program Dependencies	6
3.3	Program Partitioning, Grain Size & Scheduling	6
3.4	Program Flow Mechanisms	6
4	System Interconnect Architectures	7
4.1	Static Interconnection Networks	7
4.2	Dynamic Interconnection Networks	7

1 Theory of Parallelism & The State of Computing

Imagine you are hosting a large dinner party. If you are the only cook in the kitchen, preparing the soup, the salad, the main course, and the dessert one by one takes a very long time. This is how **sequential computing** works—one instruction after another.

Parallel processing is like hiring four chefs who work in the kitchen at the same time: one chops vegetables, one stirs the soup, one bakes the main dish, and one plates the dessert. By dividing the work, dinner is ready in a fraction of the time.

In computer science, we are forced to build parallel computers because single-processor chips have hit a physical limit—they get too hot and consume too much power if we try to make them run any faster sequentially.

1.1 Architectural Development Tracks

To understand how computers got to where they are today, we can trace three historical paths of evolution:

1. **The Processor Track (Instruction-Level Parallelism - ILP):** How does a single core execute tasks faster?

SISD (Single Chef) → Pipelining (Assembly Line)
→ Superscalar (Two Hands) → VLIW (Pre-packaged tasks)
→ SMT (Multitask while waiting) → Multi-Core (Multiple Kitchens)

Instead of just running one command at a time, modern processors use assembly lines (pipelining) where one instruction is fetched while another is decoded and a third is executed.

2. **The Memory Track (Memory Wall & Caching):** Imagine a super-fast chef who chops ingredients at lightning speed, but has to walk to a warehouse across town (the main memory) to get every single onion or potato. The chef spends 99% of their time walking, not chopping. This delay is called the **Memory Wall**. To solve this, we place small, fast baskets of ingredients directly next to the cutting board (L1, L2, L3 caches). The chef checks the baskets first; only if something is missing do they make the long trip to the warehouse.
3. **The Control Track (How instructions are triggered):**
 - **Control-Flow:** Traditional programming. A program counter points to step 1, then step 2, then step 3.
 - **Data-Flow:** No program counter. An instruction executes the exact millisecond its inputs become available. (Like a factory station that glues a label on a box as soon as both a box and a label slide down the belt).
 - **Reduction (Demand-Flow):** Work is triggered only when someone asks for a result. (Like a restaurant where cooking only begins when a customer orders a specific meal).

1.2 Theoretical Models: PRAM and VLSI

To study parallel algorithms without getting bogged down by specific hardware models, scientists use simplified mathematical models:

Definition 1.1 (PRAM (Parallel Random Access Machine))

Imagine a classroom with p students (processors) who share a single large blackboard (shared memory). Any student can read from or write to any part of the blackboard in exactly 1 unit of time.

Since multiple students might try to look at or write on the same section of the board at the same time, we have different rules for access:

- **EREW** (Exclusive Read, Exclusive Write): Only one student can look at or write to a specific board section at a time. No sharing allowed during that step.
- **CREW** (Concurrent Read, Exclusive Write): Multiple students can read from the same section at the same time, but only one student is allowed to write to it at a time.
- **CRCW** (Concurrent Read, Concurrent Write): Multiple students can read and write simultaneously. If they write to the exact same spot, we need a tie-breaker rule:
 - *Common*: All students writing to that spot must write the exact same word.
 - *Arbitrary*: One random student's write succeeds, others are discarded.
 - *Priority*: The student with the lowest roll number (highest priority) gets to write.

Definition 1.2 (VLSI Complexity Model)

Unlike mathematical models, physical microchips have space constraints. The **VLSI model** evaluates algorithms based on:

- A : The silicon area of the chip (how much physical space and wiring it needs).
- T : The execution time of the algorithm.

There is a trade-off represented by $AT^2 = \Omega(f(n))$. If you want an algorithm to execute twice as fast ($T/2$), you will need a chip with four times the wiring area ($4A$) to route all the parallel signals.

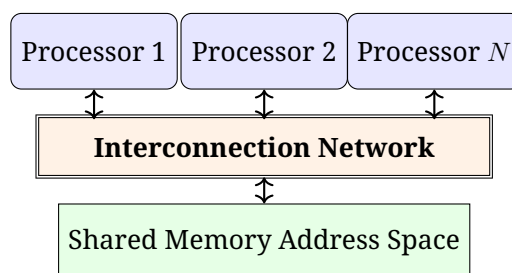
2 Parallel Computer Models

We categorize parallel computers based on how they organize their processors and memory.

2.1 Multiprocessors (Shared-Memory Systems)

In a multiprocessor system, all processors share a single, giant pool of memory. If Processor 1 writes a value to memory address 100, Processor 2 can read it instantly.

Analogy: Imagine a group of friends sitting around a single table, building one giant jigsaw puzzle together. Everyone can see and touch the entire puzzle.



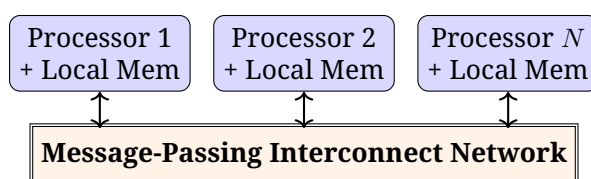
We divide shared-memory systems based on memory access speeds:

1. **UMA** (Uniform Memory Access): Every processor takes the exact same amount of time to access any memory location. (Like a circular dining table where all food platters are equally close to everyone).
2. **NUMA** (Non-Uniform Memory Access): Memory is split into local blocks near each processor. Accessing your own local memory is super fast, but accessing another processor's local memory across the chip takes longer. (Like a long banquet table where the salad is right in front of you, but you have to ask someone to pass the dessert from the far end).
3. **COMA** (Cache-Only Memory Access): There is no static main memory location. The memory behaves like a set of floating caches. Data dynamically migrates to whichever processor is currently using it. (Like sliding the pizza box to whoever is currently hungry).

2.2 Multicomputers (Distributed-Memory Systems)

In a multicomputer, each processor has its own private memory. Processor 1 cannot touch or see Processor 2's memory. To share information, they must explicitly send messages to each other over a network.

Analogy: Imagine a group of friends working on their own individual sections of a project, each in their own separate house. To coordinate, they must send each other emails or text messages.



Note. Multicomputers are also called **NORMA (No-Remote Memory Access)** systems. They scale up to thousands of computers easily because there is no single central memory bottleneck that everyone is fighting over.

2.3 Comparison Matrix

Here is a side-by-side comparison of the two systems:

2.4 Vector and SIMD Computers

- **SIMD (Single Instruction, Multiple Data):** Imagine a gym class where the instructor shouts: “Do a jumping jack!”. Everyone executes the exact same instruction at

Table 1: Multiprocessors vs. Multicomputers

Feature	Multiprocessors (Shared Memory)	Multicomputers (Distributed Memory)
Memory Address Space	Single global pool of memory.	Separate private memory for each node.
How they talk	Read and write shared variables.	Explicitly send/receive network messages.
Scalability Limits	Hard to scale (too many processors choke the memory bus).	Highly scalable (just add more independent computers).
Cache Coherence	Needs complex hardware to ensure no one reads stale data.	Not applicable (no shared variables exist).
Programming difficulty	Easier to write, but debugging race conditions is a nightmare.	Harder to write (you must partition the data yourself).

the exact same time, but they do it using their own bodies (local data). This is how modern GPUs process millions of pixels at once.

- **Multivector Computers:** Specialized machines that execute operations on large arrays (vectors) of numbers. Instead of doing one addition at a time, they feed vectors through deep arithmetic pipelines like an assembly line.

3 Program and Network Properties

3.1 Conditions of Parallelism

Before running two tasks at the same time, we must ensure they do not interfere with each other. This is formalised by **Bernstein’s Conditions**.

Let S_1 and S_2 be two statements. Let I be the variables they read (Inputs), and O be the variables they write to (Outputs).

Theorem 3.1 (Bernstein’s Conditions)

Two statements S_1 and S_2 can run safely in parallel if and only if they satisfy three simple rules:

$I_1 \cap O_2 = \emptyset$

(Rule 1: S1 must not read what S2 is writing)

(1)

$O_1 \cap I_2 = \emptyset$

(Rule 2: S2 must not read what S1 is writing)

(2)

$O_1 \cap O_2 = \emptyset$

(Rule 3: They must not write to the same location)

(3)

Example 3.1 Applying Bernstein’s Conditions

Let’s analyze three instructions to see if we can run them at the same time:

$S_1: A = X + Y$

(Reads X, Y ; Writes A)

$S_2: B = A \times 2$

(Reads A ; Writes B)

- $S_3: X = C - D$ (Reads C, D ; Writes X)

Can S_1 and S_2 run together? No. S_1 writes A , and S_2 needs to read A to do its calculation. If they run at the same time, S_2 might read the old value of A before S_1 updates it. This is a **Flow Dependency**.

Can S_1 and S_3 run together? No. S_1 needs to read X , and S_3 is updating X . If they run at the same time, S_1 might read the new value of X instead of the old one. This is an **Anti-dependency**.

3.2 Types of Program Dependencies

What stops us from running code in parallel? Five main obstacles:

- **Flow (Data) Dependency (Read-After-Write):** You cannot eat (S_2) until I finish cooking (S_1).
- **Anti-dependency (Write-After-Read):** I want to overwrite a page (S_2), but I must wait until you finish reading it (S_1) so I don't destroy your data.
- **Output Dependency (Write-After-Write):** We both try to write our names on the same line. If we do it at the same time, the text gets scribbled over.
- **Control Dependency:** You cannot decide to open your umbrella (S_2) until you evaluate whether it is raining (S_1).
- **Resource Dependency:** We both want to cook, but there is only one frying pan in the kitchen. One of us has to wait.

3.3 Program Partitioning, Grain Size & Scheduling

How do we split a large task among multiple processors?

- **Grain Size:** This is the amount of work a processor does before it needs to communicate or synchronize with others.
 - *Fine-grain (Tiny tasks):* Split the work into single sentences. Excellent parallelism, but processors spend all their time talking to each other rather than doing actual work (high communication overhead).
 - *Coarse-grain (Large tasks):* Split the work into whole chapters. Low communication overhead, but some processors might finish early and sit idle while waiting for the others (unbalanced load).
- **Scheduling:** The art of assigning these split tasks to the available cores. Good scheduling keeps all cores busy and minimizes the overhead of managing them.

3.4 Program Flow Mechanisms

How does a computer decide which instruction to run next?

1. **Control-Driven:** Sequential execution guided by a program counter (the classic von Neumann model).
2. **Data-Driven (Dataflow):** Instructions wait passively until all their operands are ready, and then they execute immediately. No program counter exists.
3. **Demand-Driven (Reduction):** The program starts from the desired result and works backward, only triggering calculations that are absolutely necessary to find that result.

4 System Interconnect Architectures

To let processors and memory modules talk to each other, we connect them using networks.

4.1 Static Interconnection Networks

In static networks, the connections are permanent copper wires or optical links.

- **Network Diameter:** The maximum number of hops a message must take to travel between the two farthest nodes in the network. (Like the worst-case travel distance in a subway system).
- **Linear Array:** Nodes connected in a straight line. Network diameter is $N - 1$ (very slow for large networks).
- **Ring:** A linear array where the ends are joined to make a circle. The diameter is cut in half to $\lfloor N/2 \rfloor$.
- **Mesh:** Nodes arranged in a grid. Diameter for an $r \times c$ grid is $(r - 1) + (c - 1)$.
- **Torus:** A grid where the edges wrap around (like a donut). If you walk off the right side, you wrap around to the left. This reduces the network diameter.
- **Hypercube:** A multi-dimensional grid. A 3D hypercube is a cube with 8 nodes. A 4D hypercube connects two 3D cubes. It is highly popular because its network diameter grows very slowly (logarithmically) as you add more nodes.

4.2 Dynamic Interconnection Networks

In dynamic networks, we use electronic switches to change paths on the fly.

- **Bus:** A single shared wire. It's cheap and simple, but if Processor 1 is using it, everyone else must wait. It does not scale well.
- **Crossbar Switch:** A matrix grid of switches connecting every input directly to every output. It's lightning fast and non-blocking, but extremely expensive to build because the cost grows quadratically ($O(N^2)$).
- **Multistage Interconnection Network (MIN):** A compromise. By using multiple layers of smaller 2×2 switches (like airports and transfer hubs), it connects everyone with a reasonable cost ($O(N \log N)$ switches). Examples include Omega and Butterfly networks.